

Optimización Paralela en GPU del Problema Extendido de la Mochila Mediante CUDA

Christian Ferrer, Sebastian Cisternas, Patricio Valdés,
Eliaser Concha y Benjamin Borgeaud
Actividad 3 — INFO1195

Junio 2026

Resumen

Este informe presenta la implementación y análisis de un algoritmo genético (AG) paralelizado en CUDA para resolver una versión extendida del problema de la mochila (knapsack), considerando múltiples restricciones duras y blandas. Se desarrollaron tres variantes: secuencial en CPU, CUDA básico y CUDA optimizado. Los experimentos se ejecutaron en dos arquitecturas de hardware diferentes para evaluar la escalabilidad: una NVIDIA RTX 4090 (servidor de la universidad) y una NVIDIA RTX 3060 (PC local). Los resultados demuestran la viabilidad empírica del modelo, manteniendo una correctitud exacta en la factibilidad, pero con diferencias de rendimiento grandes. Se reportó speed-ups hasta **161.1x** en la RTX 4090 del servidor y **49.6x** en la RTX 3060 local para instancias medias, validando así que la GPU es ideal para la evaluación paralela de funciones de aptitud complejas.

Índice

1. Introducción, Contexto y Formulación Extendida	3
1.1. El Problema de la Mochila	3
1.2. Complejidad Computacional	3
1.3. Aplicaciones Reales	3
1.4. Versión Extendida del Problema	4
1.5. Restricciones Duras vs Blandas	4
1.6. Función de Aptitud	4
2. Metodología y Diseño Experimental	5
2.1. Configuración del Algoritmo Genético	5
2.2. Variantes Implementadas	5
2.3. Instancias de Prueba	6
2.4. Diseño Experimental	6
2.5. Hardware	6
3. Descripción del Algoritmo Genético y Variantes	7
3.1. Representación de la Población	7
3.2. Generación Inicial	7
3.3. Evaluación de Aptitud	7
3.4. Selección por Torneo	8
3.5. Cruzamiento de Un Punto	8
3.6. Mutación por Flip de Bits	8
3.7. Elitismo	8
3.8. Criterio de Término	8
3.9. Diferencias entre Variantes	9
4. Resultados, Tablas y Gráficos	9
5. Resultados, Tablas y Gráficos	9
5.1. Tiempos de Ejecución y Speed-up en Servidor U (RTX 4090)	10
5.2. Tiempos de Ejecución y Speed-up en Equipo Local (RTX 3060)	10
6. Análisis Técnico y Conclusiones	10
6.1. Determinismo y Correctitud (Factibilidad Idéntica)	11
6.2. El Derrumbe de la Factibilidad en Instancias <i>Large</i>	11
6.3. Ley de Amdahl y Escalabilidad Masiva de la 4090	11
6.4. Potencia de CPU Secuencial	11

1. Introducción, Contexto y Formulación Extendida

1.1. El Problema de la Mochila

El problema clásico de la mochila (knapsack problem) consiste en seleccionar un subconjunto de items, donde cada item posee un valor y un peso, de manera que el valor total sea máximo sin exceder la capacidad de la mochila. Formalmente, sea N un conjunto finito de n items donde cada item i tiene un valor v_i y un peso w_i . La mochila posee una capacidad máxima W . Se define una variable binaria x_i para cada item:

$$x_i = \begin{cases} 1 & \text{si el item } i \text{ es seleccionado} \\ 0 & \text{si el item } i \text{ no es seleccionado} \end{cases}$$

La formulación clásica es:

$$\text{Maximizar } Z = \sum_{i=1}^n v_i x_i \quad (1)$$

$$\text{sujeto a } \sum_{i=1}^n w_i x_i \leq W \quad (2)$$

$$x_i \in \{0, 1\}, \quad i = 1, 2, \dots, n \quad (3)$$

1.2. Complejidad Computacional

El problema de la mochila 0/1 es **NP-hard**, lo que significa que no existe un algoritmo exacto con tiempo polinómico conocido para resolverlo. Para instancias grandes, los algoritmos exactos (programación dinámica, ramificación y poda) se vuelven inviables. Los algoritmos metaheurísticos, como los algoritmos genéticos, ofrecen soluciones de buena calidad en tiempo razonable.

1.3. Aplicaciones Reales

- **Inversión:** Selección de activos dentro de un presupuesto fijo
- **Corte de materiales:** Maximizar el aprovechamiento de láminas
- **Carga de contenedores:** Optimizar el peso y volumen en transporte
- **Recursos computacionales:** Asignación de tareas a servidores
- **Dieta:** Seleccionar alimentos maximizando nutrición dentro de restricciones calóricas

1.4. Versión Extendida del Problema

En esta actividad, cada ítem i posee:

- **ID:** identificador único
- **Valor:** beneficio asociado (v_i)
- **Peso:** peso ocupado (w_i)
- **Volumen:** espacio ocupado (vol_i)
- **Categoría:** grupo al que pertenece (c_i)

Y la mochila tiene restricciones adicionales:

1. **Peso máximo:** $\sum w_i x_i \leq W$ (restricción dura)
2. **Volumen máximo:** $\sum vol_i \cdot x_i \leq V$ (restricción dura)
3. **Restricciones por categoría:** mínimo/máximo de ítems por categoría (restricción blanda)
4. **Incompatibilidades:** ciertos pares de ítems no pueden coexistir (restricción dura)
5. **Dependencias:** si se selecciona un ítem, debe seleccionarse otro asociado (restricción dura)

1.5. Restricciones Duras vs Blandas

Cuadro 1: Clasificación de restricciones

Restricción	Tipo	Tratamiento
Peso máximo W	Dura	Debe cumplirse obligatoriamente
Volumen máximo V	Dura	Debe cumplirse obligatoriamente
Incompatibilidades	Dura	Debe cumplirse obligatoriamente
Dependencias	Dura	Debe cumplirse obligatoriamente
Categorías	Blanda	Penalizada en fitness, permiten violación temporal

1.6. Función de Aptitud

La función de aptitud combina el valor total con penalizaciones por restricciones incumplidas:

$$\text{fitness}(X) = \sum_{i=1}^n v_i x_i - \alpha \cdot \text{exceso_peso} - \beta \cdot \text{exceso_volumen} - \gamma \cdot \text{violaciones_cat} - \delta \cdot \text{incompat} - \varepsilon \cdot \text{deps}$$

Los valores de penalización se escalan según el tamaño de la instancia:

Cuadro 2: Valores de penalización por instancia

Parámetro	Small	Medium	Large
α (peso)	100	1,000	10,000
β (volumen)	100	1,000	10,000
γ (categoría)	50	500	5,000
δ (incompatibilidad)	1,000,000	1,000,000	1,000,000
ε (dependencia)	1,000,000	1,000,000	1,000,000

Las penalizaciones para restricciones duras (δ , ε) se fijan en 1,000,000 (“death penalty”) para garantizar que cualquier solución factible sea preferida sobre cualquier solución infeasible.

2. Metodología y Diseño Experimental

2.1. Configuración del Algoritmo Genético

Cuadro 3: Parámetros del AG

Parámetro	Valor
Tamaño de población	1,024 / 4,096 / 16,384
Número de generaciones	300 (small/medium), 100 (large)
Probabilidad de cruzamiento	0.85
Probabilidad de mutación	0.01 por gen
Tamaño del torneo	5
Elitismo	4 individuos
Tamaño de bloque CUDA	128 / 256 / 512

2.2. Variantes Implementadas

1. **CPU Secuencial:** Implementación en C++ con evaluación secuencial de fitness, selección por torneo, cruzamiento de un punto, mutación por flip de bits y elitismo.
2. **CUDA Básico:** Paralelización en GPU donde cada hilo evalúa el fitness de un individuo. La población se mantiene en memoria global durante toda la evolución.
3. **CUDA Optimizado:** Agrega shared memory para datos de items (reutilización dentro del bloque), inicialización greedy de la población y medición precisa de kernels con CUDA events.

2.3. Instancias de Prueba

Cuadro 4: Instancias de prueba

Instancia	Items	Categorías	Incompat.	Depend.	W (40 %)	V (40 %)
Small	100	5	99	3	968	629
Medium	1,000	8	99	2	20,570	16,146
Large	10,000	12	499	2	404,491	300,805

Las capacidades W y V se calcularon como el 40 % de la suma total de pesos y volúmenes, respectivamente.

2.4. Diseño Experimental

- **Repeticiones:** 10 ejecuciones por cada configuración
- **Semillas:** Registradas (42–51) para reproducibilidad
- **Combinaciones:** 3 instancias $\times$ 3 variantes $\times$ 3 poblaciones $\times$ 10 repeticiones = **270 ejecuciones**
- **Experimento adicional:** Efecto del block size (128, 256, 512) en medium con pop=4,096

2.5. Hardware

Cuadro 5: Especificaciones del hardware

Componente	Especificación
GPU	NVIDIA GeForce RTX 3060
VRAM	12 GB GDDR6
SMs	28
Compute Capability	8.6
Shared memory/bloque	48 KB
Max threads/bloque	1,024
Driver	591.86 (CUDA 13.1)
CUDA Toolkit	13.3
Compiler	nvcc + MSVC 2022

3. Descripción del Algoritmo Genético y Variantes

3.1. Representación de la Población

Cada individuo se representa como un cromosoma binario de largo n (número de items):

$$X = (x_1, x_2, x_3, \dots, x_n), \quad x_i \in \{0, 1\}$$

En la versión CPU, la población se almacena como un `vector<Chromosome>` donde cada cromosoma contiene un vector de genes. En CUDA, la población se almacena como un arreglo aplanado en memoria global: `d_genes[pop_size * n_items]`, permitiendo accesos coalescentes.

3.2. Generación Inicial

Se utiliza una estrategia híbrida con tres tipos de individuos:

1. **Greedy (40 %):** Se ordenan los items por relación valor/peso descendente y se seleccionan los primeros $\sim 25\%$ de la capacidad máxima.
2. **Semi-aleatorio (33 %):** Cada gen tiene 20% de probabilidad de ser 1.
3. **Totalmente aleatorio (27 %):** 50% de probabilidad por gen.

La inicialización greedy sesga la población hacia soluciones con menos items, lo que mejora la factibilidad en instancias con muchas incompatibilidades.

3.3. Evaluación de Aptitud

Cada hilo CUDA evalúa un individuo completo:

1. Calcula valor total, peso total y volumen total
2. Calcula penalizaciones por violaciones
3. Verifica factibilidad (restricciones duras)
4. Retorna `fitness = valor_total - penalización`

En la versión optimizada, los datos de items (valores, pesos, volúmenes, categorías) se cargan a shared memory para reutilización dentro del bloque.

3.4. Selección por Torneo

Cada individuo selecciona un ganador de un torneo de tamaño $k = 5$:

```
int tournamentSelect(pop, k, rng) {
    best = random(pop);
    for (i = 1; i < k; i++) {
        candidate = random(pop);
        if (candidate.fitness > best.fitness)
            best = candidate;
    }
    return best;
}
```

3.5. Cruzamiento de Un Punto

Se selecciona un punto de corte aleatorio. Los hijos heredan genes de ambos padres:

```
point = random(1, n-1);
child1[i] = (i < point) ? parent1[i] : parent2[i];
child2[i] = (i < point) ? parent2[i] : parent1[i];
```

Probabilidad de cruzamiento: 0.85. Si no se cruzan, los padres se copian directamente.

3.6. Mutación por Flip de Bits

Cada gen tiene probabilidad 0.01 de invertirse:

```
for (i = 0; i < n; i++)
    if (random() < mutation_rate)
        gene[i] = 1 - gene[i];
```

3.7. Elitismo

Se preservan los 4 individuos con mayor fitness de cada generación, copiándolos directamente a la siguiente población sin pasar por selección/cruzamiento/mutación.

3.8. Criterio de Término

El algoritmo ejecuta un número fijo de generaciones (300 para small/medium, 100 para large).

3.9. Diferencias entre Variantes

Cuadro 6: Comparación de variantes

Aspecto	CPU	CUDA Básico	CUDA Opt
Evaluación fitness	Secuencial	1 hilo/individuo	1 hilo/individuo + shared mem
Selección por torneo	Secuencial	1 torneo/hilo	1 torneo/hilo
Cruzamiento	Secuencial	1 crossover/2 hilos	1 crossover/2 hilos
Mutación	Secuencial	1 hilo/individuo	1 hilo/individuo
Inicialización	Greedy + random	Greedy + random	Greedy + random
Elitismo	CPU search	CPU search	CPU search
Medición	clock()	cudaEvent	cudaEvent

4. Resultados, Tablas y Gráficos

Para evaluar la solidez del algoritmo y su escalabilidad, las pruebas principales se ejecutaron en el entorno oficial de cuda de la universidad, utilizando posteriormente un equipo local como línea base para la comparativa de hardware:

- **Servidor Universidad (Entorno Principal):** GPU NVIDIA RTX 4090 (24GB VRAM, 16384 CUDA Cores) — CPU Intel i9-14900KF, 125GB DDR5.
- **Equipo Local (Comparativa):** GPU NVIDIA RTX 3060 (12GB VRAM, 3584 CUDA Cores) — CPU Ryzen 5 5600X, 32GB RAM.

Los resultados se obtuvieron con 10 repeticiones por configuración (semillas 42–51).

5. Resultados, Tablas y Gráficos

Para evaluar la solidez del algoritmo y su escalabilidad, las pruebas principales se ejecutaron en el entorno oficial de la universidad, utilizando posteriormente un equipo local como línea base para la comparativa de hardware:

- **Servidor Universitario (Entorno Principal):** GPU NVIDIA RTX 4090 (24GB VRAM, 16384 CUDA Cores) — CPU Intel i9-14900KF, 125GB DDR5.
- **Equipo Local (Comparativa):** GPU NVIDIA RTX 3060 (12GB VRAM, 3584 CUDA Cores) — CPU Ryzen 5 5600X, 32GB RAM.

Los resultados se obtuvieron con 10 repeticiones por configuración (semillas 42–51).

5.1. Tiempos de Ejecución y Speed-up en Servidor U (RTX 4090)

Cuadro 7: Resultados Servidor Universitario (RTX 4090 - i9-14900KF)

Instancia	Población	CPU (ms)	CUDA Base (ms)	CUDA Opt (ms)	Speed-up (Opt)	Factible
Small	1,024	387 $\pm$ 2	63 $\pm$ 2	65 $\pm$ 0	6.0x	55.4 %
	4,096	1,626 $\pm$ 12	70 $\pm$ 0	73 $\pm$ 1	22.4x	55.7 %
	16,384	6,813 $\pm$ 53	100 $\pm$ 2	101 $\pm$ 1	67.6x	56.2 %
Medium	1,024	3,497 $\pm$ 18	311 $\pm$ 3	309 $\pm$ 0	11.3x	36.7 %
	4,096	15,186 $\pm$ 49	328 $\pm$ 1	326 $\pm$ 0	46.6x	36.6 %
	16,384	62,574 $\pm$ 192	397 $\pm$ 1	388 $\pm$ 0	161.1x	35.5 %
Large	1,024	19,846 $\pm$ 335	1,478 $\pm$ 6	1,474 $\pm$ 1	13.5x	0.9 %
	4,096	79,994 $\pm$ 1,292	1,994 $\pm$ 1	1,994 $\pm$ 1	40.1x	0.3 %
	16,384	328,125 $\pm$ 4,125	4,685 $\pm$ 3	4,684 $\pm$ 2	70.1x	0.1 %

5.2. Tiempos de Ejecución y Speed-up en Equipo Local (RTX 3060)

Cuadro 8: Resultados Equipo Local (RTX 3060 - Ryzen 5 5600X)

Instancia	Población	CPU (ms)	CUDA Base (ms)	CUDA Opt (ms)	Speed-up (Opt)	Factible
Small	1,024	606 $\pm$ 10	105 $\pm$ 4	106 $\pm$ 2	5.7x	55.4 %
	4,096	2,686 $\pm$ 32	113 $\pm$ 1	121 $\pm$ 6	22.2x	55.7 %
	16,384	11,447 $\pm$ 103	400 $\pm$ 3	398 $\pm$ 3	28.7x	56.2 %
Medium	1,024	7,798 $\pm$ 73	536 $\pm$ 5	531 $\pm$ 2	14.7x	36.7 %
	4,096	33,094 $\pm$ 107	708 $\pm$ 1	668 $\pm$ 1	49.6x	36.6 %
	16,384	132,015 $\pm$ 211	3,172 $\pm$ 3	2,980 $\pm$ 4	44.3x	35.5 %
Large	1,024	27,569 $\pm$ 419	2,320 $\pm$ 6	2,327 $\pm$ 6	11.8x	0.9 %
	4,096	113,597 $\pm$ 1,667	3,396 $\pm$ 3	3,390 $\pm$ 2	33.5x	0.3 %
	16,384	518,126 $\pm$ 6,995	14,273 $\pm$ 8	14,262 $\pm$ 12	36.3x	0.1 %

Se observa que la versión CUDA elimina gran parte de la variabilidad del tiempo de ejecución, reportando desviaciones estándar casi nulas en el servidor universitario en comparación con las oscilaciones presentadas por las CPU en cargas secuenciales largas.

6. Análisis Técnico y Conclusiones

El análisis de los resultados experimentales confirma varias teorías arquitectónicas sobre la GPU y la naturaleza combinatoria de la mochila extendida.

6.1. Determinismo y Correctitud (Factibilidad Idéntica)

La métrica más reveladora de la implementación no es el tiempo, sino la **columna de Factibilidad**. Independientemente de si la ejecución ocurre en un i9, un Ryzen 5, una RTX 4090 o una RTX 3060, las tasas de factibilidad son matemáticamente idénticas (por ejemplo, 56.2 % en Small, pop=16384 o 35.5 % en Medium). Esto corrobora que:

- No existen condiciones de carrera (*race conditions*) durante las mutaciones o cruza-mientos en la GPU.
- Los estados aleatorios locales por hilo en CUDA (*curandState*) están correctamente desfasados y gestionados.
- Las penalizaciones y evaluaciones de restricciones (peso, volumen, dependencias e incompatibilidades) están implementadas simétricamente en CPU y CUDA.

6.2. El Derrumbe de la Factibilidad en Instancias *Large*

En la instancia de 10,000 ítems, el porcentaje de factibilidad colapsa a menos del 1 %. Esto no representa un error en el código, sino el comportamiento teórico esperado. El problema de la mochila extendida es **NP-hard**; al pasar de 100 ítems a 10,000 ítems, el espacio de búsqueda escala de 2^{100} a 2^{10000} . La probabilidad de que un individuo generado (o mutado) al azar respete simultáneamente miles de restricciones cruzadas y dependencias sin el uso de heurísticas de reparación focalizadas es astronómicamente baja. El algoritmo de castigo funciona a la perfección depurando soluciones inválidas.

6.3. Ley de Amdahl y Escalabilidad Masiva de la 4090

Los resultados evidencian un patrón claro: **el speed-up crece sustancialmente junto al tamaño de la población.**

- **La RTX 4090 escala magistralmente hasta alcanzar 161.1x de speed-up.** Gracias a sus 16,384 *CUDA cores* y un abrumador ancho de banda de 1 TB/s, cuando se utilizan poblaciones grandes (16,384) la GPU es capaz de evaluar a los individuos literalmente en un solo ciclo masivo. Evaluaciones que tardan más de 1 minuto en el robusto i9 (62,574 ms) se resuelven en 388 ms.
- Como línea base comparativa, en la RTX 3060 el speed-up se estabiliza cerca del 44x para esa misma población grande, demostrando cómo una arquitectura con menos núcleos sufre mayor impacto en cargas masivas.
- Con una población pequeña (1,024 individuos), en ambos escenarios la transferencia de memoria *Host-to-Device* oculta el beneficio de los cálculos masivos paralelos de la GPU, limitando el speed-up a valores mucho menores (11.3x en la 4090 y 14.7x en la 3060 para la instancia media).

6.4. Potencia de CPU Secuencial

La comparativa de las cargas de CPU secuencial confirma la abrumadora superioridad del servidor. El i9-14900KF de la universidad es consistentemente $\sim 2x$ más rápido que el Ryzen 5 5600X. Esto obedece a un mayor IPC, frecuencias que rozan los 6.0 GHz y la ventaja latente en la latencia ofrecida por la memoria RAM DDR5 de 125GB.

Conclusiones Finales

El desarrollo de este AG paralelo mediante CUDA confirma empíricamente la ventaja de la GPU en la resolución de problemas con alta carga de restricciones cruzadas.

1. **Aceleración Histórica:** Lograr reducciones desde 518 segundos (CPU local) a apenas 4.6 segundos (CUDA servidor 4090) para 16,384 soluciones en una instancia de 10,000 ítems demuestra que la evaluación de *fitness* encaja naturalmente en el modelo de hardware SIMT (*Single Instruction, Multiple Threads*).
2. **Limitaciones Combinatorias:** Si bien CUDA permite procesar inmensas poblaciones en tiempo récord, el colapso de factibilidad a $\sim 0.1\%$ subraya que el poder bruto no sustituye el diseño de operadores. Para escalar a *Large*, el algoritmo debe madurar hacia un diseño de reparación y mutación inteligente, no sólo delegar el volumen de evaluación.
3. **Coherencia y Determinismo:** La correspondencia absoluta en la proporción de soluciones factibles encontradas independientemente de la máquina (Intel, AMD, Ada Lovelace o Turing) avala la calidad del código base y valida totalmente la experimentación.